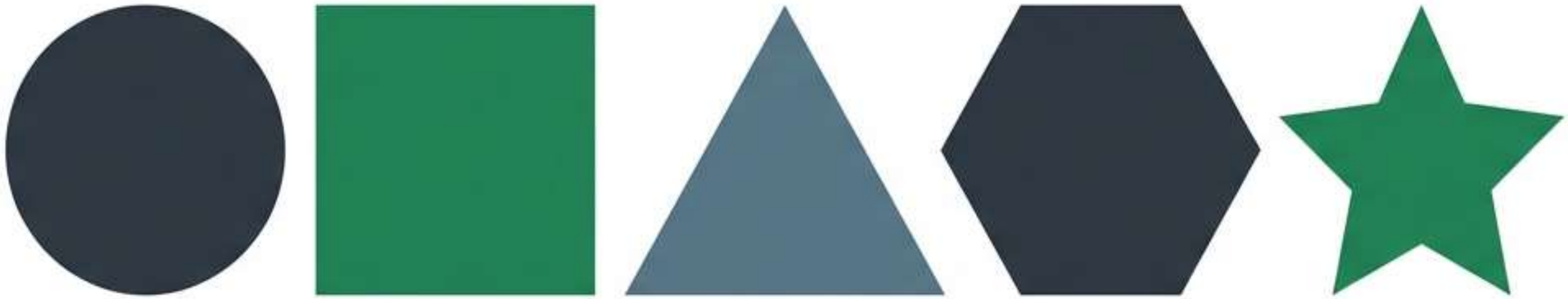
Testleaf
Always Ahead

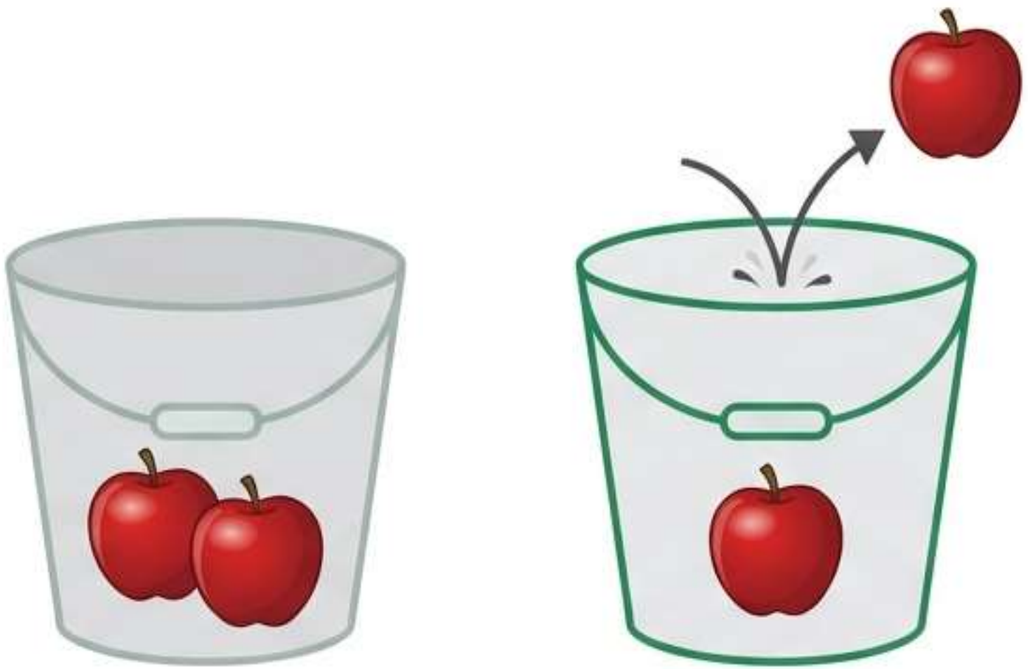


Java Collections Framework: The Set Interface

Managing Unique Data Structures in Selenium WebDriver

Overview: The Set Interface

- A specific collection that models the mathematical set abstraction.
- **Key Constraint:** It contains **NO duplicate elements.**
- **Structure:** An Interface (cannot be instantiated directly).
- **Relevance:** Essential for handling unique window handles in Selenium.

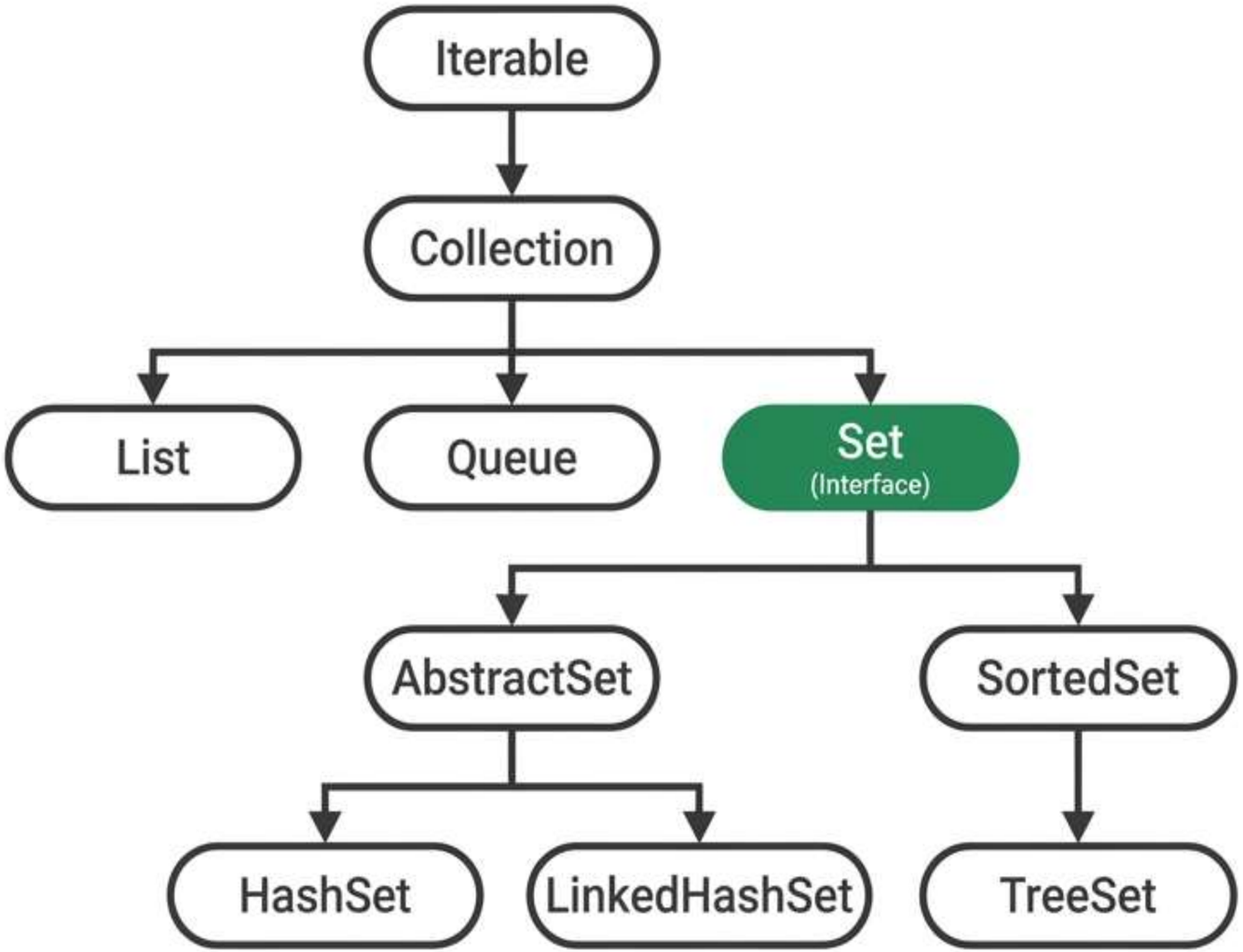


List

Set

Set cannot have duplicate values

The Collections Hierarchy



Implementation Classes & Ordering Behaviour

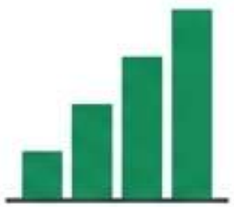
HashSet

Uses Hashing Algorithm.

Stores elements in Random/Unpredictable order.

Concept: Hashing Algorithm (Random Order)

TreeSet



Uses Red-Black Tree.

Stores elements in Natural Sorting order.

Concept: ASCII order

LinkedHashSet

Uses Hash Table + Linked List.

Maintains Insertion Order

Concept: insertion order

Syntax and Instantiation

Testleaf
Always Ahead

Interface (Contract)

Generic (Type Safety)

```
Set<String> mentors = new HashSet<String>();
```

Object Reference

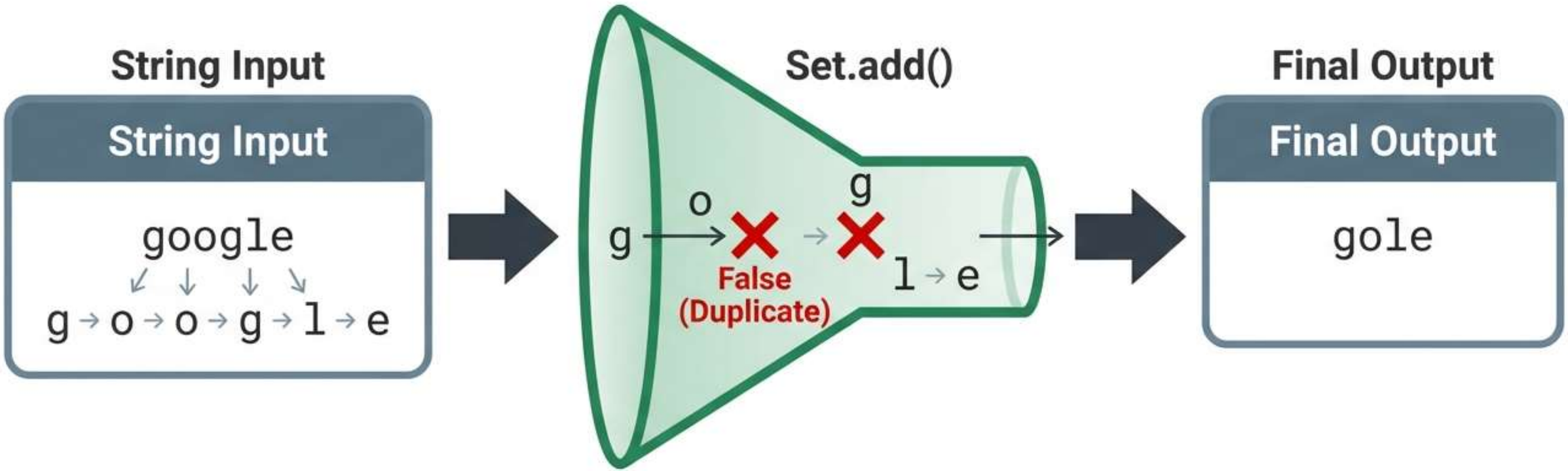
Memory Allocation

Implementation Class

Pro Tip

We use the **Interface** as the reference type (Upcasting) to allow polymorphism, restricting access to only methods defined in the Set interface.

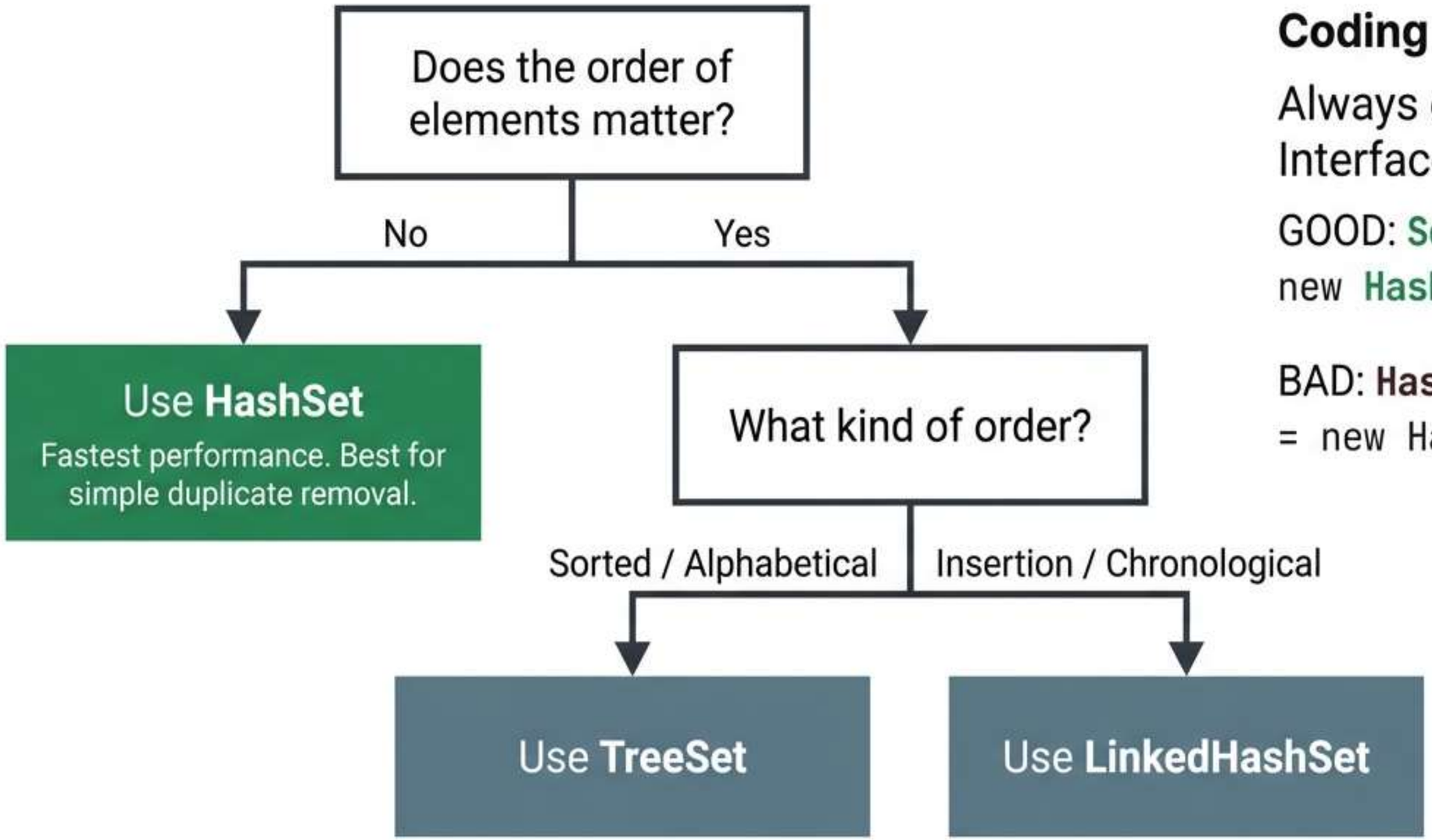
Practical Application: Removing Duplicates




```
Set<Character> uniqueChars = new LinkedHashSet<>(); // Output: gole
```

Best Practices: Strategic Selection

Testleaf
Always Ahead



Coding Standard

Always code to the Interface:

GOOD: `Set<String> data = new HashSet<>();`

BAD: `HashSet<String> data = new HashSet<>();`

Critical Interview Concepts

Testleaf
Always Ahead

List vs. Set

List allows duplicates & index access. Set enforces uniqueness & has no index.

Internal Mechanics

HashSet uses a **HashMap** internally. It stores your element as the "**Key**" and a dummy object as the "**Value**".

Sorting Logic

To sort unique items: Add them to a **TreeSet** (natural sort) or convert **Set** to **List** and use **Collections.sort()**.

Scenario

Use **LinkedHashSet** for logs or history where the **sequence of events must be preserved**.

Key Takeaways

- **Uniqueness:** `Set` is the primary tool for eliminating duplicates.
- **Polymorphism:** Cannot instantiate `'Set'` directly; must use implementation classes.
- **Ordering Rules:** Remember `Hash` (Random), `Tree` (Sorted), `LinkedHash` (Insertion).
- **Access:** No index-based access (no `get(i)`); use Iterators or For-Each loops.
- **Syntax:** Always use `Generics` <Type> to ensure type safety.